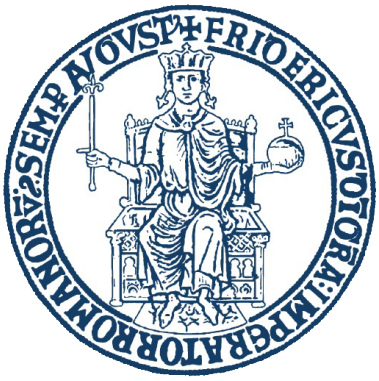


UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Conquista del Territorio (Traccia B) - Progetto LSO 2025

Novembre 2026

Autore:
Lorenzo Trignano (N86004496)

Indice

1 Introduzione **3**

2 Architettura di Rete e Gestione della Concorrenza **4**
2.1 Gestione della Scalabilità Orizzontale 4

3 Protocollo di Comunicazione: Design e Semantica **5**
3.1 Scelta del Formato Testuale 5

4 Logica di Gioco e Astrazione (Fog of War) **6**
4.1 Scelte di implementazione della Logica: 6

5 Gestione dello Stato e Robustezza delle Sessioni **7**
5.1 Strutture Dati Centralizzate: 7
5.2 Gestione delle disconnessioni: 7

6 Implementazione del Client **8**

7 Containerizzazione e Deployment **9**
7.1 Dockerfile: 9
7.2 Docker Compose: 9

8 Conclusioni e Scalabilità **10**
8.1 Valutazione delle Scelte: 10
8.2 Evoluzione del Sistema: 10

1 Introduzione

Questo progetto fornisce un'implementazione del gioco "Conquista del Territorio" giocabile in modalità multiplayer su rete. Il progetto è composto da un Server centrale che gestisce la mappa e le interazioni tra i giocatori, e da un Client che permette agli utenti di connettersi al server, esplorare la mappa e conquistare territori.

L'intera applicazione è containerizzata utilizzando Docker e Docker Compose per facilitare il setup e il deployment.

2 Architettura di Rete e Gestione della Concorrenza

La scelta fondamentale nell'implementazione del server è stata l'utilizzo di un approccio multi-threaded (tramite la libreria `pthread`), affiancato da un demone per le comunicazioni asincrone.

Motivazioni della scelta:

- **Gestione Asincrona:** Il gioco richiede l'invio periodico di una mappa globale da parte del server, operazione che deve avvenire in background senza bloccare la ricezione dei comandi.
- **Semplicità di Sincronizzazione:** L'uso di un singolo Mutex POSIX permette di proteggere in modo efficace tutte le strutture dati globali (come la mappa e l'array dei giocatori), prevenendo *race conditions*.
- **Reattività:** Il server è in grado di rispondere immediatamente grazie a un thread dedicato per ogni client connesso.

2.1 Gestione della Scalabilità Orizzontale

L'architettura permette di parametrizzare la porta di ascolto in fase di avvio. L'isolamento dei thread garantisce che il server possa gestire molteplici connessioni simultanee senza colli di bottiglia causati dall'I/O bloccante, rendendo il servizio facilmente scalabile.

3 Protocollo di Comunicazione: Design e Semantica

Il protocollo è di tipo testuale, orientato ai comandi, facilitando il debug e la scalabilità.

Comandi implementati:

- `REG <nick>:<pass>`: Inizializza una nuova sessione di gioco per l'utente, inserendolo nella griglia.
- `MOV <direzione>`: Trasmette la direzione della mossa (U, D, L, R).
- `LIST`: Richiede l'elenco degli utenti attualmente online.
- `QUIT`: Segnala la disconnessione volontaria dalla partita.

3.1 Scelta del Formato Testuale

Si è optato per un protocollo "Human-Readable" invece di uno binario per diverse ragioni strategiche:

- **Flessibilità del Parsing:** I comandi testuali permettono una validazione immediata tramite stringhe, facilitando l'estensione del protocollo con nuovi comandi.
- **Robustezza della Rete:** L'uso del carattere di newline per delimitare i messaggi garantisce una facile e sicura frammentazione dei pacchetti sullo stream TCP.

4 Logica di Gioco e Astrazione (Fog of War)

La logica di conquista e gestione della *Fog of War* è isolata interamente nel server.

4.1 Scelte di implementazione della Logica:

- **Mappa Globale:** Il server mantiene una matrice bidimensionale immutabile contenente la reale dislocazione fisica degli ostacoli e delle celle libere.
- **Dominio Territoriale:** Una matrice globale separata traccia gli indici dei giocatori che attualmente possiedono ciascuna cella libera.
- **Visione Privata (Fog of War):** Ogni giocatore possiede una propria matrice degli ostacoli (`known_walls`) che viene aggiornata dinamicamente dal server solo in seguito ad una collisione esplorativa. Ciò garantisce che i muri restino ignoti fino alla loro scoperta individuale.

5 Gestione dello Stato e Robustezza delle Sessioni

5.1 Strutture Dati Centralizzate:

Il server mantiene lo stato del gioco tramite strutture dati mirate:

- **Array dei Client:** Traccia il file descriptor, lo stato di autenticazione, la posizione, il punteggio e la matrice privata dei muri di ogni utente.
- **Sincronizzazione (Mutex):** L'aggiornamento simultaneo delle celle conquistate e la generazione dei pacchetti di broadcast sono racchiusi in sezioni critiche per garantire la totale consistenza dei dati.

5.2 Gestione delle disconnessioni:

Una scelta implementativa critica riguarda la robustezza: se un giocatore si disconnette, il server chiude in modo sicuro il socket e libera lo slot per nuovi utenti, ma mantiene inalterato il dominio sulle celle precedentemente conquistate sulla mappa, fino al passaggio di un nuovo sfidante.

6 Implementazione del Client

Il client è progettato per essere il più "snello" possibile (Thin Client).
È volutamente privato di logica decisionale per tre motivi:

- **Veridicità dello Stato:** Il server è l'unica "Source of Truth". Inviando stringhe pre-formattate si annulla la possibilità che un client presenti uno stato diverso da quello reale del server.
- **Semplicità di Aggiornamento:** Modificando la logica di gioco o la durata sul server, tutti i client beneficiano del cambiamento istantaneamente senza dover aggiornare il software locale.
- **Interfaccia Non-Bloccante:** L'uso di un'architettura multi-thread anche lato client (con un thread interamente dedicato all'ascolto su socket) assicura che il programma rimanga reattivo agli eventi periodici di rete mentre l'utente sta digitando una mossa.

7 Containerizzazione e Deployment

L'intero ecosistema è configurato per essere eseguito in ambienti isolati tramite Docker.

7.1 Dockerfile:

- **Immagine Base:** È stata scelta l'immagine ufficiale `ubuntu:latest` per garantire un ambiente di compilazione GCC standard e affidabile.
- **Compilazione Isolata:** Il Dockerfile copia i sorgenti e compila il server all'interno del container, garantendo che i binari siano compatibili con l'ambiente di esecuzione e liberi da dipendenze esterne.

7.2 Docker Compose:

- **Networking ed Esecuzione:** Viene definito un servizio per il game server, configurando automaticamente il port-mapping (`8080:8080`). Questo permette al client nativo di connettersi al demone virtualizzato in modo immediato tramite localhost, facilitando esponenzialmente il testing e l'orchestrazione.

8 Conclusioni e Scalabilità

L'implementazione ha dimostrato che un'architettura basata su thread concorrenti è la scelta ottimale per un gioco esplorativo in tempo reale, garantendo stabilità ed efficienza.

8.1 Valutazione delle Scelte:

- **Efficienza:** La delega della logica di gioco al server e la scomposizione in thread alleggeriscono il carico del client e abbattono le latenze di I/O.
- **Integrità:** La gestione sicura tramite un singolo lock centralizzato elimina alla radice il rischio di corruzione dei dati durante il ribaltamento delle proprietà territoriali.
- **Centralizzazione:** Il modello "Thin Client" assicura che il server mantenga il totale controllo normativo sulla partita e sul timeout.

8.2 Evoluzione del Sistema:

Grazie alla progettazione modulare e alla containerizzazione con Docker, il sistema è intrinsecamente pronto per il deployment in cloud. L'infrastruttura è facilmente scalabile e configurabile per ambienti di rete complessi, rendendo possibili future estensioni (es. salvataggio su database, Thread Pool) senza dover rivoluzionare l'architettura base.